

Vypracování úloh – Lineární a celočíselné programování

Úkol 1 – Grafické řešení úlohy LP

Maximalizujeme

$$z = -17x_1 + 40x_2$$

za podmínek

$$x_1 + 2x_2 \geq 180,$$

$$2x_1 \leq 240,$$

$$x_1 - 3x_2 \geq -120,$$

$$x_1, x_2 \geq 0.$$

Hraniční přímky a oblast přípustných řešení

Označme

$$L_1 : x_1 + 2x_2 = 180, \quad L_2 : x_1 = 120, \quad L_3 : x_1 - 3x_2 = -120.$$

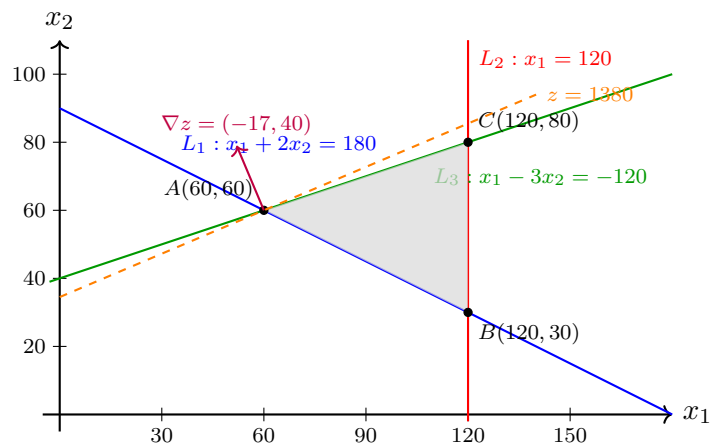
Najdeme průsečíky těchto přímek a otestujeme přípustnost:

- $L_1 \cap L_3$: ze soustavy $x_1 + 2x_2 = 180$, $x_1 - 3x_2 = -120$ odečtením dostaneme $5x_2 = 300$, tedy $x_2 = 60$ a $x_1 = 60$. Bod $A = (60, 60)$. Kontrola: $60 \leq 120 \checkmark$.
- $L_1 \cap L_2$: $x_1 = 120 \Rightarrow 2x_2 = 60 \Rightarrow x_2 = 30$. Bod $B = (120, 30)$. Kontrola L_3 : $120 - 90 = 30 \geq -120 \checkmark$.
- $L_2 \cap L_3$: $x_1 = 120 \Rightarrow 120 - 3x_2 = -120 \Rightarrow x_2 = 80$. Bod $C = (120, 80)$. Kontrola L_1 : $120 + 160 = 280 \geq 180 \checkmark$.

Ostatní průsečíky se souřadnicovými osami leží mimo přípustnou oblast (např. $(0, 90)$ porušuje L_3 , $(0, 40)$ porušuje L_1 , $(120, 0)$ porušuje L_1). Oblast přípustných řešení je tedy uzavřený trojúhelník $\triangle ABC$ s vrcholy

$$A = (60, 60), \quad B = (120, 30), \quad C = (120, 80).$$

Grafické znázornění



Hodnoty účelové funkce v základních (krajních) řešeních

Vrcholy přípustného polytopu jsou *základní přípustná řešení*:

Vrchol	(x_1, x_2)	$z = -17x_1 + 40x_2$
A	$(60, 60)$	$-1020 + 2400 = \mathbf{1380}$
B	$(120, 30)$	$-2040 + 1200 = -840$
C	$(120, 80)$	$-2040 + 3200 = 1160$

Optimální řešení

Maximum je dosaženo v jediném vrcholu

$$x_1^* = 60, \quad x_2^* = 60, \quad z^* = 1380.$$

Že jde o jediné optimum, ověříme směrův: gradient $\nabla z = (-17, 40)$. Hrana $A \rightarrow B$ má směr $(60, -30) \sim (2, -1)$ se skalárním součinem $-17 \cdot 2 + 40 \cdot (-1) = -74 < 0$ (tj. z klesá). Hrana $A \rightarrow C$ má směr $(60, 20) \sim (3, 1)$ se skalárním součinem $-51 + 40 = -11 < 0$ (z rovněž klesá). Žádná hrana z A není kolmá na ∇z , tedy alternativní optimum neexistuje a optimum je jednoznačné.

Úkol 2 – Pivovar

a) Proměnné a model

Proměnné:

- $x_1 \geq 0 \dots$ litry světlého piva za týden,
- $x_2 \geq 0 \dots$ litry polotmavého piva za týden,
- $x_3 \geq 0 \dots$ litry tmavého piva za týden.

Účelová funkce (týdenní tržby v Kč):

$$z = 27x_1 + 29x_2 + 35x_3 \rightarrow \max.$$

Omezení. Spotřeba sladu je $0,2x_1 + 0,25x_2 + 0,28x_3$ kg a nesmí překročit zásobu 150 kg. Spotřeba chmele $3x_1 + 2x_2 + 2,3x_3$ g nesmí překročit $2\text{ kg} = 2000$ g. Pracovní čas brigádníka je omezen 40 hodinami týdně. Kvasnic je prakticky neomezené množství (nezavádí omezení), nicméně pro úplnost lze formálně psát $1 \cdot x_1 + 1,5x_2 + 2x_3 \leq M$ s $M \rightarrow \infty$. Záporná výroba nemá smysl.

$$\begin{array}{rcll} \text{(slad)} & 0,20x_1 & + & 0,25x_2 & + & 0,28x_3 & \leq & 150, \\ \text{(chmel)} & 3x_1 & + & 2x_2 & + & 2,3x_3 & \leq & 2000, \\ \text{(čas)} & 0,05x_1 & + & 0,04x_2 & + & 0,06x_3 & \leq & 40, \\ & x_1, x_2, x_3 & & & & & \geq & 0. \end{array}$$

b) Optimální řešení

Úlohu řešíme simplexovou metodou (resp. libovolným LP řešičem, viz Appendix A). Optimum:

$$x_1^* = \frac{10750}{19} \approx 565,79, \quad x_2^* = 0, \quad x_3^* = \frac{2500}{19} \approx 131,58, \quad z^* = \frac{377750}{19} \approx 19881,58 \text{ Kč.}$$

c) Zbylé suroviny

Dosažením optima:

$$\text{slad: } 0,2 \cdot 565,79 + 0,28 \cdot 131,58 = 113,158 + 36,842 = 150 \Rightarrow \text{zbývá } 0 \text{ kg,}$$

$$\text{chmel: } 3 \cdot 565,79 + 2,3 \cdot 131,58 = 1697,37 + 302,63 = 2000 \Rightarrow \text{zbývá } 0 \text{ g,}$$

$$\text{čas: } 0,05 \cdot 565,79 + 0,06 \cdot 131,58 = 28,289 + 7,895 = 36,184 \Rightarrow \text{zbývá cca } 3,82 \text{ h.}$$

Aktivní (vyčerpaná) jsou tedy omezení sladu a chmele; čas je *neaktivní*, brigádník nepracuje na plno.

d) Hodina práce navíc

Protože omezení času není v optimu aktivní (zbývá $\approx 3,82$ h volných), **prodloužení pracovní doby o jednu hodinu nepřinese žádné dodatečné tržby**. Stínová cena (duální proměnná) omezení času je v optimu nulová,

$$\frac{\partial z^*}{\partial b_{\text{čas}}} = 0,$$

takže zvýšení kapacity času z 40 na 41 hodin ponechá optimální plán i hodnotu z^* beze změny.

e) Fixní náklady na zavedení výroby

Zavedeme binární proměnné $y_i \in \{0, 1\}$ pro $i \in \{1, 2, 3\}$ s významem „druh i se vůbec vyrábí“. Pro každou položku přidáme *velké-M* omezení provazující x_i s y_i :

$$x_i \leq M_i y_i, \quad i = 1, 2, 3,$$

kde M_i je dostatečně velká horní mez možné produkce (např. $M_i = \min\{150/a_{1i}, 2000/a_{2i}, 40/a_{4i}\}$, tedy individuální maximum dané surovinami). Účelová funkce se sníží o fixní náklady:

$$z = 27x_1 + 29x_2 + 35x_3 - 400(y_1 + y_2 + y_3) \rightarrow \max.$$

Z LP úlohy se tak stává smíšeně celočíselné programování (MILP).

f) Maximálně dva typy piva

Použijeme tytéž binární proměnné y_i s vazbami $x_i \leq M_i y_i$ a přidáme omezení, že lze „zapnout“ nejvýše dvě:

$$y_1 + y_2 + y_3 \leq 2, \quad y_i \in \{0, 1\}.$$

Účelová funkce zůstává $z = 27x_1 + 29x_2 + 35x_3$ (bez fixních nákladů), pokud body e) a f) neuvažujeme současně.

Úkol 3 – Distribuce počítačů

a) Proměnné a model

Označme střediska $i \in \{1, 2, 3\} = \{\text{Plzeň, Pardubice, Olomouc}\}$ a odběratele $j \in \{1, 2, 3, 4\} = \{\text{Brno, Praha, Ostrava, Liberec}\}$. Definujeme

$$x_{ij} \in \mathbb{Z}_{\geq 0} \dots \text{počet jízd dodávky (kapacita 60 ks) z } i \text{ do } j.$$

Matice nákladů na jednu jízdu (tis. Kč):

$$C = \begin{pmatrix} 11 & 4 & 17 & 9 \\ 6 & 7 & 10 & 8 \\ 3 & 9 & 5 & 12 \end{pmatrix}.$$

Účelová funkce (celkové náklady v tis. Kč):

$$z = \sum_{i=1}^3 \sum_{j=1}^4 c_{ij} x_{ij} \rightarrow \min.$$

Omezení. Kapacita každého střediska v ks: počet vyvezených ks z i je $60 \sum_j x_{ij}$ a nesmí překročit $K_i \in \{400, 200, 250\}$:

$$60 \sum_{j=1}^4 x_{ij} \leq K_i \quad \text{pro } i = 1, 2, 3.$$

Požadavky odběratelů $D_j \in \{180, 250, 160, 110\}$ musí být uspokojeny (dodáno alespoň tolik ks):

$$60 \sum_{i=1}^3 x_{ij} \geq D_j \quad \text{pro } j = 1, 2, 3, 4.$$

Celočíselnost a nezápornost: $x_{ij} \in \mathbb{Z}_{\geq 0}$.

b) Optimální řešení

Řešením celočíselné úlohy (viz Appendix B) je matice plánu jízd

$$X^* = \begin{pmatrix} 0 & 5 & 0 & 1 \\ 2 & 0 & 0 & 1 \\ 1 & 0 & 3 & 0 \end{pmatrix} \quad (\text{řádky: Plzeň, Pardubice, Olomouc; sloupce: Brno, Praha, Ostrava, Liberec}).$$

Celkové náklady:

$$z^* = 5 \cdot 4 + 1 \cdot 9 + 2 \cdot 6 + 1 \cdot 8 + 1 \cdot 3 + 3 \cdot 5 = 20 + 9 + 12 + 8 + 3 + 15 = \mathbf{67} \text{ tis. Kč.}$$

Kontrola dodávek a využití kapacit:

Odběratel:	Brno	Praha	Ostrava	Liberec	Středisko:	Plz.	Pard.	Olom.
Dodáno (ks):	180	300	180	120	Vyvezeno (ks):	360	180	240
Požadavek:	180	250	160	110	Kapacita:	400	200	250

Všechny požadavky jsou splněny, žádná kapacita není překročena. (Některé dodávky jsou vyšší než smluvní minimum, což plyne z diskretnosti dodávek po 60 ks.)

c) Řidič z Pardubic na Moravu max. jednou měsíčně

Řidič z Pardubic ($i = 2$) jede na Moravu právě tehdy, když cílí do Brna ($j = 1$) nebo do Ostravy ($j = 3$). Podmínka má tvar

$$x_{2,1} + x_{2,3} \leq 1.$$

d) Fixní poplatek 450 tis. Kč za každou použitou trasu

Pro každou dvojici (i, j) zavedeme indikátor $y_{ij} \in \{0, 1\}$ s významem „trasa $i \rightarrow j$ je použita“. Vazbu mezi x_{ij} a y_{ij} zajistíme přes „velké M “:

$$x_{ij} \leq M_{ij} y_{ij}, \quad y_{ij} \in \{0, 1\}, \quad \forall i, j,$$

kde lze volit $M_{ij} = \lceil K_i/60 \rceil$ (maximální možný počet jízd ze střediska i). Účelovou funkci pak rozšíříme o fixní poplatky:

$$z = \sum_{i,j} c_{ij} x_{ij} + 450 \sum_{i,j} y_{ij} \rightarrow \min.$$

Úloha tím přechází z čistě celočíselného transportního programu na MILP s pevnými náklady (*fixed-charge transportation problem*).

Úkol 4 – TSP heuristika pro uy734

a) Popis algoritmu

Použili jsme dvoustupňovou heuristiku:

Krok 1: Nearest Neighbor (NN) – konstrukce počátečního turné. Začneme v libovolném startovním městě (zvolíme město č. 0). V každém kroku z aktuálního města v navštívíme nejbližší dosud nenavštívené město u :

$$u = \arg \min_{w \in V \setminus T} d(v, w),$$

kde d je euklidovská vzdálenost a T je dosud sestavené turné. Po navštívení všech měst se vrátíme do startu. Časová složitost je $O(n^2)$.

Krok 2: 2-opt – lokální vylepšování. Mějme turné $\pi = (v_0, v_1, \dots, v_{n-1}, v_0)$. Pro každou dvojici hran (v_i, v_{i+1}) a (v_j, v_{j+1}) s $i + 1 < j$ uvažujeme *2-opt swap*: hrany odstraníme a nahradíme je hranami (v_i, v_j) a (v_{i+1}, v_{j+1}) , což odpovídá obrácení segmentu $v_{i+1}, \dots, v_j$. Pokud

$$\Delta = d(v_i, v_j) + d(v_{i+1}, v_{j+1}) - d(v_i, v_{i+1}) - d(v_j, v_{j+1}) < 0,$$

swap provedeme. Iterujeme, dokud existuje zlepšující výměna (*first improvement* nebo *best improvement*). Algoritmus konverguje do 2-optimálního turné, ze kterého žádná dvojhranová výměna nedokáže snížit délku.

Vzdálenosti. Standardní formát TSPLIB pro uy734 je EUC_2D – vzdálenosti se zaokrouhlují na nejbližší celé číslo (`int`). Pro porovnání s optimem (79 114) je nutné počítat délky stejně.

b) Srovnání s optimem

Pro instanci uy734 (734 měst v Uruguayi) je známé optimum

$$L^{\text{opt}} = 79\,114.$$

Konkrétní výsledky našeho běhu (Python, start v městě č. 0, *first-improvement* 2-opt do úplné konvergence – viz Appendix C):

metoda	délka L	relativní mezera $(L - L^{\text{opt}})/L^{\text{opt}}$	čas
Nearest Neighbor (start 0)	99 247	25,45 %	0,01 s
NN + 2-opt	84 269	6,52 %	0,85 s

Samotný Nearest Neighbor je tedy o čtvrtinu horší než optimum, což je obvyklé. Po aplikaci 2-opt klesne mezera pod 7 %, přičemž algoritmus skončí v lokálním (2-)optimu, ze kterého žádná dvojhranová výměna nedokáže turné dále zkrátit. Konkrétní hodnota závisí na startovním městě a varianty 2-optu (*first* vs. *best improvement*), případně na restartování z více NN řešení; pro porovnání s optimem se počítá s EUC_2D (zaokrouhlené euklidovské vzdálenosti).

Appendix A – Skript pro Úkol 2 (LP)

```
1 from scipy.optimize import linprog
2
3 # max 27 x1 + 29 x2 + 35 x3 -> min -27 x1 - 29 x2 - 35 x3
4 c = [-27, -29, -35]
5
```

```

6      # Omezení:  slad, chmel, cas
7      A = [[0.20, 0.25, 0.28],      # slad    <= 150 kg
8          [3.00, 2.00, 2.30],      # chmel   <= 2000 g
9          [0.05, 0.04, 0.06]]      # cas     <= 40 h
10     b = [150, 2000, 40]

11
12     res = linprog(c, A_ub=A, b_ub=b, bounds=[(0, None)] * 3, method="highs")
13
14     print("Optimalni vyroba (l):", res.x)
15     print("Trzby (Kc):", -res.fun)
16     print("Zbyly slad (kg):", 150 - sum(A[0][i] * res.x[i] for i in range(3)))
17     print("Zbyly chmel (g):", 2000 - sum(A[1][i] * res.x[i] for i in range(3)))
18     print("Zbyly cas (h):", 40 - sum(A[2][i] * res.x[i] for i in range(3)))

```

Apendix B – Skript pro Úkol 3 (MILP)

```

1      import numpy as np
2      from scipy.optimize import milp, LinearConstraint, Bounds
3
4      # x_ij ... pocet jizd ze strediska i do odberatele j (60 ks na jizdu)
5      costs = np.array([[11, 4, 17, 9],      # Plzen
6                       [ 6, 7, 10, 8],      # Pardubice
7                       [ 3, 9,  5,12]])      # Olomouc
8      caps  = [400, 200, 250]                # ks
9      dems   = [180, 250, 160, 110]          # ks
10
11     c = costs.flatten().astype(float)
12
13     A_rows, b_ub = [], []
14     # kapacita: 60 * sum_j x_ij <= caps_i
15     for i in range(3):
16         row = [0] * 12
17         for j in range(4):
18             row[i * 4 + j] = 60
19         A_rows.append(row); b_ub.append(caps[i])
20
21     # pozadavky: 60 * sum_i x_ij >= dems_j -> -60 sum <= -dems_j
22     for j in range(4):
23         row = [0] * 12
24         for i in range(3):
25             row[i * 4 + j] = -60
26         A_rows.append(row); b_ub.append(-dems[j])
27
28     A_ub = np.array(A_rows)
29     constr = LinearConstraint(A_ub, -np.inf, b_ub)
30
31     res = milp(c, constraints=constr, integrality=np.ones(12),
32               bounds=Bounds(lb=0, ub=np.inf))
33
34     X = res.x.reshape(3, 4)
35     print("Plan jizd:\n", X.astype(int))
36     print("Naklady (tis. Kc):", res.fun)
37     print("Vyvezeno (ks):", (X.sum(axis=1) * 60).astype(int))
38     print("Dodano (ks):", (X.sum(axis=0) * 60).astype(int))

```

Apendix C – Skript pro Úkol 4 (TSP heuristika)

```

1      import urllib.request, math, time
2

```

```

3 URL = "http://www.math.uwaterloo.ca/tsp/world/uy734.tsp"
4
5 def load_tsplib(url):
6     """Stáhne EUC_2D TSPLIB instanci a vrati seznam (x, y)."""
7     raw = urllib.request.urlopen(url).read().decode()
8     coords, in_section = [], False
9     for line in raw.splitlines():
10        s = line.strip()
11        if s.startswith("NODE_COORD_SECTION"):
12            in_section = True; continue
13        if s in ("EOF", ""):
14            if in_section: break
15            else: continue
16        if in_section:
17            parts = s.split()
18            coords.append((float(parts[1]), float(parts[2])))
19        return coords
20
21 def dist(a, b):
22     """TSPLIB EUC_2D: zaokrouhlení na nejbližší celé číslo."""
23     return int(round(math.hypot(a[0] - b[0], a[1] - b[1])))
24
25 def tour_length(tour, coords):
26     n = len(tour)
27     return sum(dist(coords[tour[i]], coords[tour[(i + 1) % n]]) for i in range(n))
28
29 def nearest_neighbor(coords, start=0):
30     n = len(coords)
31     visited = [False] * n
32     tour = [start]; visited[start] = True
33     for _ in range(n - 1):
34         cur = tour[-1]
35         best, best_d = -1, float("inf")
36         for j in range(n):
37             if not visited[j]:
38                 d = dist(coords[cur], coords[j])
39                 if d < best_d:
40                     best, best_d = j, d
41         tour.append(best); visited[best] = True
42     return tour
43
44 def two_opt(tour, coords):
45     """First-improvement 2-opt do úplné konvergence."""
46     n = len(tour)
47     improved = True
48     while improved:
49         improved = False
50         for i in range(n - 1):
51             a, b = tour[i], tour[i + 1]
52             for j in range(i + 2, n):
53                 c = tour[j]
54                 d = tour[(j + 1) % n]
55                 if i == 0 and (j + 1) % n == 0:
56                     continue
57                 delta = (dist(coords[a], coords[c]) + dist(coords[b], coords[d])
58                     - dist(coords[a], coords[b]) - dist(coords[c], coords[d]))
59                 if delta < 0:
60                     tour[i + 1:j + 1] = tour[i + 1:j + 1][::-1]
61                     improved = True
62             break
63         if improved:
64             break

```

```

65     return tour
66
67     if __name__ == "__main__":
68         coords = load_tsplib(URL)
69         print(f"Nacteno {len(coords)} mest.")
70
71         t0 = time.time()
72         nn_tour = nearest_neighbor(coords, start=0)
73         print(f"NN delka      = {tour_length(nn_tour, coords):>7d}"
74               f"    ({time.time() - t0:.1f}s)")
75
76         t0 = time.time()
77         opt_tour = two_opt(nn_tour[:], coords)
78         print(f"NN+2opt delka = {tour_length(opt_tour, coords):>7d}"
79               f"    ({time.time() - t0:.1f}s)")
80
81         print("Optimum (uy734) = 79114")

```